

Automation Framework

Login & New Test Case Features — Playwright + TypeScript

Playwright

TypeScript

Page Object Model

UI + API Tests

Runs Offline

29 Test Cases

29

TOTAL TESTS

16

SMOKE TESTS

19

UI TESTS

10

API TESTS

2

BROWSERS

0

EXTERNAL
DEPENDENCIES



1. Overview



This framework provides automated test coverage for two core features — **Login** and **New Test Case** — covering both UI interactions and API contracts.

It is built with **Playwright** and **TypeScript**, follows the **Page Object Model** pattern, and is designed to run **fully offline** with no external dependencies, no real backend, and no credentials required out of the box.

The suite covers positive and negative scenarios across both UI and API layers, matching the test cases defined in the project's Test Cases document (TC-001) and aligned with the framework rules defined in **rules.md**.



2. Architecture & Project Structure



```
playwright-suite/
```

```

app/      ← Local mock HTML pages (runs offline)
config/   ← Environment config (base URLs, credentials)
data/     ← Test data factories + user credentials
fixtures/ ← Custom Playwright fixtures (loginPage,
authenticatedPage)
mocks/    ← Route interception handlers for UI tests
pages/    ← Page Object Model classes
utils/    ← API client, network capture helper
tests/

  e2e/login/ login.positive.spec.ts
              login.negative.spec.ts
  e2e/new-test-case/ new-test-case.positive.spec.ts
                    new-test-case.negative.spec.ts
  api/ login.api.spec.ts
        new-test-case.api.spec.ts

mock-api-server.js ← Real Node.js mock API server (port 4001)
playwright.config.ts ← Framework configuration
rules.md          ← Coding standards and architecture rules

```

Each feature has its own folder. Positive and negative scenarios are always in separate spec files. No test logic lives in page objects — no page logic lives in spec files.



3. How It Runs Offline

The framework requires no real application, no real backend, and no credentials to run. This is achieved through two layers:

LAYER 1 — LOCAL MOCK APP (PORT 4000)

- Static HTML pages served by npx serve
- Implements the exact form behaviour from the requirements
- Frontend validation, error messages, redirects all work as in the real app
- Started automatically by Playwright before tests run

LAYER 2 — MOCK API SERVER (PORT 4001)

- Real Node.js HTTP server using no external libraries
- Handles POST /login and POST /test-cases
- Returns correct status codes: 200, 201, 400, 401
- Also started automatically by Playwright before tests run

npm test



Playwright starts both servers



Tests run against localhost



HTML + JUnit reports generated

When a real application is available, a `.env` file with `BASE_URL` and `API_BASE_URL` is all that is needed to switch — no code changes required.



4. Page Object Model



Every page has a dedicated class that encapsulates all locators and actions. Spec files never interact with the DOM directly — they only call Page Object methods.

PAGE OBJECTS

- **BasePage** — shared `navigate()`, `waitForNetworkIdle()`
- **LoginPage** — `login()`, `fillEmail()`, `isErrorBannerVisible()` etc.
- **NewTestCasePage** — `createTestCase()`, `fillSteps()`, `isTitleErrorVisible()` etc.

LOCATOR PRIORITY (FROM RULES.MD)

- ARIA role + name — first choice
- Label text — form fields
- Placeholder — fallback for inputs
- `data-testid` — for error messages
- CSS / XPath — last resort only, with comment



5. Test Structure & Coverage



FEATURE	UI POSITIVE	UI NEGATIVE	API POSITIVE	API NEGATIVE	TOTAL
Login	3	5	2	3	13
New Test Case	4	4	3	3	16
Total	7	9	5	6	29

SMOKE TAG (@SMOKE) — 16 TESTS

- Run on every build via `npm run test:smoke`
- Cover all critical happy paths and key failure states
- Answer the question: is this build shippable?

TEST DATA

- Randomised via `@faker-js/faker` factory functions
- Fixed payloads used where order/value matters
- Credentials stored in `data/users.ts`, never hardcoded in specs

**6. Configuration****BROWSERS**

- Chromium (Desktop Chrome)
- WebKit (Desktop Safari)
- Every test runs on both automatically

TIMEOUTS

- Action timeout: 10 seconds
- Navigation timeout: 30 seconds
- Retries: 2 on CI, 0 locally

REPORTS GENERATED

- HTML report — `reports/html/`
- JUnit XML — `reports/junit/results.xml`
- Live list output in terminal

FAILURE ARTIFACTS

- Screenshot on failure
- Video on first retry
- Trace on first retry

**7. Running the Suite****COMMAND****WHAT IT RUNS**`npm test`

Full suite — 29 tests on both browsers

`npm run test:smoke`

Smoke tests only — 16 tests, fastest check

`npm run test:e2e`

UI tests only

`npm run test:api`

API tests only

`npm run test:headed`

Full suite with browser visible

COMMAND	WHAT IT RUNS
npm run test:report	Open HTML report in browser

To connect to a real application: create a `.env` file with `BASE_URL` and `API_BASE_URL`. The local servers are skipped automatically when these are set.